📗 MS11 — Service de détection d'anomalies

Documentation détaillée — données, fonctionnement du ML, benchmark et utilisation

Transport Scolaire Intelligent — PFE SDIA · Meknès · Sprint S12 (anticipé)

---

1. À quoi sert MS11 ?

MS11 **repère automatiquement les trajets de bus « anormaux »** — ceux perturbés par un **incident** (panne, accident, route bloquée, retard contextuel, surcharge d'élèves) — pour **alerter l'administration**.

> **Lien avec MS12 (prédiction ETA).** MS12 prédit la durée **normale** d'un trajet mais **ne peut pas prédire** un incident (imprévisible au départ). MS11 fait le complément : sur des trajets **déjà effectués**, il **détecte** ceux qui étaient anormaux. La boucle data-science est bouclée.

---

2. La notion d'« anomalie » et le choix du modèle

2.1 Apprentissage non supervisé

Les incidents sont **rares** (~1 %) et hétérogènes. Plutôt qu'un classifieur supervisé, on apprend à quoi ressemble un trajet **normal** et on signale les écarts. **Le label `incident_type` n'est JAMAIS utilisé pour entraîner** — seulement pour **évaluer**. C'est la démarche honnête standard en détection d'anomalies.

2.2 Isolation Forest — comment ça marche (simplement)

On coupe l'espace **au hasard** avec des arbres. Un point **anormal** (isolé) est séparé en **très peu de coupes** ; un point normal (au milieu de la foule) en demande beaucoup. Le **score d'anomalie** = facilité d'isolement.

> Analogie : dans une foule, repérer une personne unique (très grande, en rouge) prend **peu de questions**.

---

3. Les données — un dataset PROPRE à MS11

Important : MS11 a son **propre dataset** (`ms11-analytique/data/anomaly_dataset.csv`, généré par `dataset_anomaly.py`), **indépendant de MS12**. Il est conçu pour rendre la détection **non triviale** : les anomalies sont **hétérogènes et contextuelles** (sinon un simple seuil suffirait et le ML serait inutile).

3 natures d'anomalie injectées (~1,25 % des trajets, split temporel 30 j de test) :

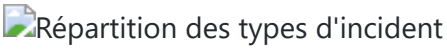
Type	Description	Ce qui le trahit
franc (panne/accident/route_bloquee)	durée énorme (4–12× la base)	le `ratio` (facile)
contextuel (retard)	ratio modéré (2–3×) MAIS par **conditions légères** (trafic fluide)	la `surprise` (ratio ajusté au contexte)
surcharge	durée quasi normale mais **montées/descentes anormales** (22–36)	`dwell` / `nb_montees`

→ **Aucune variable unique ne capte les 3 natures** : c'est ce qui justifie un détecteur **multivarié**.

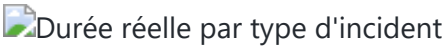
Base réelle : `duree_base_s` = durée routière OSRM de Meknès (repli vol d'oiseau si OSRM off).

Visualisation des données

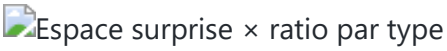
Répartition des types — la grande majorité des trajets sont normaux ; les incidents sont rares et de 3 natures.

 Répartition des types d'incident

Durée réelle par type — les incidents « francs » (panne/accident) **explosent** la durée ; « contextuel » et « surcharge » restent **proches du normal** → impossibles à distinguer par la seule durée.

 Durée réelle par type d'incident

Où vivent les anomalies — dans le plan `surprise × ratio`, chaque nature occupe une zone différente : les **francs** à droite (ratio élevé), les **contextuels** en haut (surprise élevée), les **surcharges** près de l'origine (invisibles ici → ce sont les montées/ `dwell` qui les trahissent).

 Espace surprise × ratio par type

---

4. Feature engineering (`features.py`) — le vrai enjeu

Le choix du **modèle** compte moins que la **construction des bons signaux**. Trois signaux clés :

Feature	Formule / rôle	Capte
`ratio`	durée réelle / durée base OSRM	anomalies **franches**
`surprise`	`ratio / ratio_attendu`, où ratio_attendu vient du **contexte** (trafic × météo × ramadan × souk)	anomalies **contextuelles** (ratio élevé alors que conditions légères)
`dwell`, `nb_montees`, `nb_descentes`, `taux_occupation`	activité aux arrêts	anomalies de **surcharge**

L'idée de la `surprise` : un ratio de 2,4 est **normal** en embouteillage+pluie (le contexte le justifie), mais **anormal** par temps fluide. On divise donc le ratio par ce qui est *attendu vu le contexte*. `incident_type` reste **exclu** des features (label d'évaluation).

Jeu de features « curé » du détecteur retenu : `[surprise, ratio, dwell, nb_montees, nb_descentes, taux_occupation]`.

Distributions des 3 signaux (normal vs incident, échelle log) — le `ratio` seul rate les anomalies discrètes ; la `surprise` récupère les contextuelles ; le `dwell` isole les surcharges. **Aucun signal seul ne suffit** → c'est l'argument central pour un modèle multivarié.

 Distributions des signaux clés

Corrélations des features curées — `ratio` et `surprise` sont liés mais **distincts** ; `dwell`/montées apportent une information **orthogonale** (donc utile à combiner).

 Corrélations des features curées

---

5. Benchmark honnête des détecteurs (`train_anomaly.py`)

5.1 Protocole

- **Split temporel** (30 derniers jours = test), entraînement **non supervisé** (sans labels).
- **Budget d'alerte fixe** (on signale les *contamination %* trajets les plus anormaux) → comparaison équitable.
- Métriques : **PR-AUC** (adaptée au déséquilibre), ROC-AUC, F1, précision, rappel.

5.2 Résultats (gelés dans `data/anomaly_benchmark.json`)

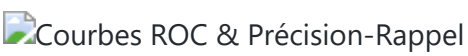
Détecteur	PR-AUC	ROC-AUC	F1	Précision	Rappel
`iforest_curated` 🏆	**0.91**	**1.00**	**0.78**	0.70	**0.88**
`seuil_surprise` (univarié)	0.80	0.99	0.63	0.55	0.75
`seuil_ratio` (univarié)	0.79	0.94	0.67	0.60	0.75
`iforest_all` (toutes features, **dilué**)	0.48	0.98	0.44	0.40	0.50

> Confusion du gagnant (test) : `[[709 TN, 3 FP], [1 FN, 7 TP]]` → 7/8 incidents détectés.

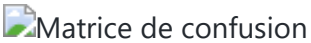
Comparaison visuelle (PR-AUC) — le détecteur curé domine nettement les seuils univariés et l'IF « dilué ».

 Benchmark PR-AUC

Courbes ROC & Précision-Rappel (indépendantes du seuil) — `iforest_curated` est au-dessus partout.


Courbes ROC & Précision-Rappel

Matrice de confusion du gagnant (budget d'alerte fixe) — 7/8 incidents détectés, 3 fausses alertes.


Matrice de confusion

5.3 Interprétation — la triple leçon (à défendre)

1. **Le feature engineering prime.** La `surprise` (contextuelle) sépare mieux que le `ratio` brut (`seuil_surprise` ROC-AUC 0.99 > `seuil_ratio` 0.94).
2. **La curation prime.** `iforest_curated` (0.91) **écrase** `iforest_all` (0.48) : trop de features **diluent** le signal — Isolation Forest traite toutes les dimensions à égalité.
3. **Le multivarié gagne — et c'est mérité.** Comme les anomalies sont **hétérogènes** (franc + contextuel
    - surcharge), **aucun seuil univarié** ne les capte toutes ; le détecteur curé les **combine** → PR-AUC 0.91.

> Contraste avec MS12 (où le modèle simple gagnait) : ici, le problème est **réellement multivarié**, donc le modèle plus riche est justifié. **On ne suppose rien : on benchmarke et on tranche par les chiffres.**

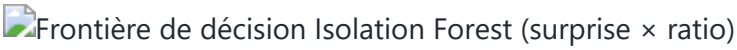
5.4 Le modèle gelé

`detecteur = iforest_curated` · features `[surprise, ratio, dwell, nb_montees, nb_descentes, taux_occupation]` · seuil 0,615 · contamination 0,013. Artefacts : `anomaly_model.pkl`, `anomaly_config.json`, `anomaly_benchmark.json`.

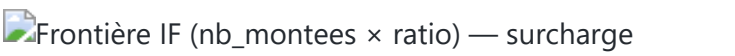
5bis. Voir le modèle non supervisé (Isolation Forest)

Ces vues rendent le fonctionnement du modèle **visuel** — utile pour la soutenance.

Frontière de décision (`surprise × ratio`) — on entraîne un IF sur ces 2 axes et on colore le score partout : fond **clair** = zone « normale », fond **foncé** = zone « anormale ». Les incidents (rouge) tombent dans le foncé → le modèle a bien appris la forme du normal.


Frontière de décision Isolation Forest (surprise × ratio)

Autre coupe (`nb_montees × ratio`) — ici on **voit** la **surcharge** (beaucoup de montées, durée quasi normale) devenir anormale : un axe qu'un simple seuil sur le ratio **ignore totalement**.


Frontière IF (nb_montees × ratio) — surcharge

Projection PCA (6 features → 2D) colorée par le score d'anomalie du modèle complet : les incidents (cerclés de rouge) se concentrent dans les zones à **score élevé** → le modèle les **isole** correctement même dans l'espace multivarié.


Projection PCA colorée par le score

Distribution des scores (normal vs incident) avec le **seuil** d'alerte : une bonne séparation = peu de recouvrement à droite du seuil (les rares chevauchements = les fausses alertes / incidents ratés).



---

6. Comment on l'utilise (service `main.py`)

Sécurité **JWT ADMIN** (même secret que MS7). CORS : Gateway uniquement. Route déjà câblée `/api/analytique/**` → 8091.

Route	Auth	Rôle
`GET /api/analytique/health`	public	service up + détecteur chargé
`GET /api/analytique/model-info`	public	métriques (PR-AUC/ROC/F1), features, seuil
`POST /api/analytique/anomalies`	**JWT ADMIN**	un trajet observé → anomalie ?
`GET /api/analytique/anomalies/scan?limit=`	**JWT ADMIN**	scanne le dataset → top anomalies + précision/rappel

Déroulé d'une détection (`POST /anomalies`)

Entrée : `duree_reelle_min` + `duree_base_s` (ou coords → OSRM) + contexte (trafic, météo, montées…). → le service calcule `ratio`, `surprise`, `dwell`… → Isolation Forest curé → `{ is_anomaly, anomaly_score, seuil, ratio_reel_base, niveau }` (niveau : normal / suspect / critique).

Exemples réels (validés) :

Trajet	Contexte	Résultat
4 min / base 3,6 min	fluide	✅ normal
8,6 min (ratio 2,4)	**embouteillage + pluie**	✅ normal (le contexte justifie)
6,5 min, **30 montées**	fluide	⚠️ **anomalie** (surcharge)
42 min (ratio 11,7)	fluide	🔴 **anomalie critique** (panne)

→ Le détecteur **ne se laisse pas piéger** par un trajet lent mais justifié (embouteillage), et attrape une **surcharge** que le ratio seul manquerait. C'est la valeur ajoutée du multivarié.

Scan global (`GET /anomalies/scan`)

Passe tout le dataset et renvoie les top anomalies + bilan vs incidents réels (≈ **précision 0,70 · rappel 0,72** sur l'ensemble ; top anomalies = accidents/pannes/surcharges réels). Prépare un futur **dashboard KPI**.

---

7. Reproduire de A à Z

```
conda activate nlp                # Python 3.11
cd ms11-analytique
pip install -r requirements.txt

python dataset_anomaly.py         # -> data/anomaly_dataset.csv (anomalies hétérogènes)
python train_anomaly.py           # benchmark + gel : anomaly_model.pkl / _config.json / _benchmark.json
uvicorn main:app --port 8091      # service

# notebook (démarche complète, figures) :
python notebooks/build_notebooks.py
jupyter nbconvert --to notebook --execute --inplace notebooks/04_anomaly_detection.ipynb
```

---

8. Notebook (démarche CRISP-DM)

`notebooks/04_anomaly_detection.ipynb` — markdown avant chaque cellule, **12 figures** dans `notebooks/figures/` :

A. EDA détaillée — répartition des types d'incident, durée par type (violon log), distributions des 3 signaux (ratio/surprise/dwell, normal vs incident), corrélations des features curées, nuage `surprise × ratio` par type.

B. Voir le modèle non supervisé — frontière de décision Isolation Forest en 2D (surprise × ratio, puis nb_montees × ratio pour la surcharge), **projection PCA** (6D→2D) colorée par le score d'anomalie, **distribution des scores** normal vs incident avec le seuil.

C. Benchmark — barres PR-AUC, **courbes ROC & Précision-Rappel**, matrice de confusion du gagnant.

Palette **accessible aux daltoniens** (Okabe-Ito pour les catégories, séquentielle *viridis* pour les scores).

---

9. Architecture & fichiers

```
ms11-analytique/                    (port 8091, /api/analytique/**)
├── main.py             ← FastAPI + JWT : health / model-info / anomalies / anomalies/scan
├── config.py           ← port, jwt_secret, dataset local, contamination
├── dataset_anomaly.py  ← génère le dataset PROPRE (anomalies contextuelles hétérogènes)
├── features.py         ← ratio, SURPRISE (contextuel), dwell… ; incident_type EXCLU
├── anomaly.py          ← détecteur unifié (seuil | IsolationForest curé), persistance joblib
├── train_anomaly.py    ← benchmark 4 détecteurs + évaluation vs label + gel du gagnant
├── osrm_client.py      ← durée/distance de base (option coords)
├── data/               ← anomaly_dataset.csv, anomaly_model.pkl, anomaly_config.json, anomaly_benchmark.json
├── notebooks/          ← 04_anomaly_detection.ipynb + figures/
├── requirements.txt · Dockerfile · .env
```

Réutilise le pattern service de MS12 (config pydantic, `verify_token` JWT, chargement joblib agnostique, Dockerfile, générateur de notebooks). **Gateway** : route déjà présente → aucune modif.

---

10. Limites & perspectives (honnêteté scientifique)

- **Données synthétiques** (ancrées OSRM) : validité réelle à confirmer sur trajets GPS terrain (perspective). Les anomalies sont **modélisées** de façon réaliste (3 natures) mais restent conçues par nous.
- **Détection a posteriori** : après le trajet. Évolution : détection **temps réel** au fil du GPS (MS3).
- **Dashboard KPI** (S12) : à brancher sur `/anomalies/scan`.

📂 Références

- Code : `ms11-analytique/*.py` · Artefacts : `ms11-analytique/data/anomaly_*.{csv,pkl,json}`
- Notebook : `ms11-analytique/notebooks/04_anomaly_detection.ipynb`
- Doc ETA : `docs/MS12_PREDICTION_ETA.md` · Résumé projet : `CLAUDE.md` (section MS11)